ZOWE SIMULATION

ZOWE:

Modern enterprises continue to rely heavily on IBM mainframe systems for mission-critical workloads such as banking transactions, airline reservations, and government records. However, traditional mainframe interaction mechanisms are complex and not well suited for cloud-native or AI-driven systems.

Zowe is an open-source framework introduced to modernize access to IBM z/OS mainframes. It provides command-line interfaces (CLI), REST APIs, and web-based tools that allow developers to interact with mainframe resources using familiar modern paradigms.

The primary objective of Zowe is to act as an **abstraction layer** between modern applications and legacy mainframe subsystems such as **CICS, DB2, and IMS**, enabling interoperability without modifying existing mainframe applications.

ZOWE ARCHITECTURE:

At a high level, Zowe follows a layered architecture:

1. **Zowe CLI (Client Layer)**
   This is the user-facing interface where commands are issued.
2. **Zowe Core Services (Middleware Layer)**
   Responsible for authentication, routing, plugin invocation, and request orchestration.
3. **z/OS Management Facility (z/OSMF)**
   Provides REST-based access to z/OS resources.
4. **Mainframe Subsystems (Backend Layer)**
   Includes CICS, DB2, IMS, datasets, and batch processing services.

ARCHITECTURE FLOW:

User Command

↓

Zowe CLI

↓

Zowe Core Services

↓

z/OSMF / Subsystem APIs

↓

IBM Mainframe Subsystems

What is a Plugin?

A plugin is a small add-on module that extends the functionality of a main system without changing the core system.

A plugin:

- Runs inside a host system
- Uses host system APIs
- Handles specific tasks
- Can be installed, updated, or removed independently

What Is a Plugin in Zowe?

A plugin is a module that enables Zowe to interact with a specific IBM mainframe subsystem.

Zowe core itself does not know how to talk to:

- CICS
- DB2
- IMS

That knowledge lives inside plugins.

Zowe Without Plugins:

Without plugins, Zowe can:

- Authenticate
- Route requests
- Manage CLI

But it cannot:

1. List CISC (Customer Information Control System) programs
2. Query DB2
3. Access IMS(Information Management System)

Plugins make this possible.

Plugin Characteristics:

- Installed locally via CLI
- Encapsulate subsystem-specific logic
- Translate CLI commands into REST calls
- Return structured responses

Examples of Plugins:

- **CICS Plugin** – for transaction and program management
- **DB2 Plugin** – for database metadata and queries
- **IMS Plugin** – for hierarchical transaction management

CICS Plugin – Data Flow Analysis and Simulation

For all CICS commands, the core flow remains the same:

User Command

↓

Zowe CLI

↓

Zowe CICS Plugin

↓

z/OSMF / CICS Management Interface (CMCI)

↓

CICS Subsystem (Region / Program / Transaction)

↓

Response (Metadata / Status)

Zowe CISC plugin commands:

1. **zowe cics list regions :**
   Lists all available CICS regions and their operational status.

   Data Flow:
   CLI
   ↓
   Zowe CICS Plugin
   ↓
   z/OSMF REST API
   ↓
   CICS Management Interface (CMCI)
   ↓
   CICS Region Control Table
   ↓
   Region Metadata

   Data Returned:
   - Regions
   - Status (Inactive/Active)

Simulated Data Flow:

Simulated CLI

↓

Mock CICS Plugin

↓

Mock Region Registry (JSON)

↓

JSON Response

Simulated Output:


```
{
  "regions": [
    { "region": "CICSREG1", "status": "ACTIVE" },
    { "region": "CICSREG2", "status": "INACTIVE" }
  ]
}
```


2. **zowe cics list programs**
   Lists programs deployed in a given CICS region.

   Real Data Flow:
   CLI
   ↓
   Zowe CICS Plugin
   ↓
   CMCI
   ↓
   CICS Program Control Table
   ↓
   Program Metadata

   Data Returned

- Program name
- Language (COBOL)
- Execution status (ENABLED / DISABLED)

Simulated Data Flow:

CLI
↓
Mock CICS Plugin
↓
Mock Program Table (JSON)
↓
JSON Response

Simulated Output:


```
{
  "programs": [
    { "name": "CUSTPGM", "language": "COBOL", "status": "ENABLED" },
    { "name": "ACCTPGM", "language": "COBOL", "status": "DISABLED" }
  ]
}
```


3. **zowe cics list transactions**
   Lists transaction codes handled by CICS.

   Real Data Flow:
   CLI
   ↓
   Zowe CICS Plugin
   ↓
   CMCI
   ↓
   CICS Transaction Definition Table
   ↓
   Transaction Metadata

   Data Returned

- Transaction ID
- Program mapping
- Transaction status

Simulated Data Flow:
CLI
↓
Mock CICS Plugin
↓
Mock Transaction Table (JSON)
↓
JSON Response

Simulated Output:


```
{
  "transactions": [
    { "txn_id": "TXN01", "program": "CUSTPGM", "status": "ACTIVE" },
    { "txn_id": "TXN02", "program": "ACCTPGM", "status": "INACTIVE" }
  ]
}
```


4. **zowe cics start program**
   Enables a CICS program so it can execute transactions.

   Real Data Flow:
   CLI
   ↓
   Zowe CICS Plugin
   ↓
   CMCI
   ↓
   CICS Program Control
   ↓
   Program State Change (DISABLED → ENABLED)

   Data Effect:

- No data retrieval
- System state update

Simulated Data Flow:
CLI
↓
Mock CICS Plugin
↓
Update Program Status in JSON Store
↓
Confirmation Response

Simulated Output:


```
{
  "program": "ACCTPGM",
  "previous_status": "DISABLED",
  "current_status": "ENABLED",
  "message": "Program started successfully"
}
```

5. **zowe cics stop program**
   Disables a CICS program to prevent execution.

   Real Data Flow:
   CLI
   ↓
   Zowe CICS Plugin
   ↓
   CMCI
   ↓
   CICS Program Control
   ↓
   Program State Change (ENABLED → DISABLED)

   Simulated Data Flow:
   CLI
   ↓
   Mock CICS Plugin
   ↓
   Update Program Status in JSON Store
   ↓
   Confirmation Response

   Simulated Output:


```
{
  "program": "CUSTPGM",
  "previous_status": "ENABLED",
  "current_status": "DISABLED",
  "message": "Program stopped successfully"
}
```


DB2 Plugin – Data Flow Analysis and Simulation

What is the DB2 Plugin?

The DB2 plugin in Zowe enables interaction with IBM DB2, a relational database management system used on IBM mainframes to store structured enterprise data.

The plugin allows users to:

- Discover databases and tables
- Execute SQL queries
- Retrieve structured data in a modern format

Zowe does not replace **DB2** but it **translates modern CLI requests into DB2-compatible operations** and returns structured responses.

General Data Flow Pattern (DB2 Commands):

User Command

↓

Zowe CLI

↓

Zowe DB2 Plugin

↓

DB2 REST Services / DB2 SQL Engine

↓

DB2 Catalog or User Tables

↓

Structured Result Set

Zowe DB2 Plugin Commands:

1. **zowe db2 list databases**
   Lists all available DB2 databases and their metadata.

   Real Data Flow:
   CLI
   ↓
   Zowe DB2 Plugin
   ↓
   DB2 REST Service
   ↓
   DB2 System Catalog
   ↓
   Database Metadata

   Data Returned:
   - Database Name
   - Owner
   - Encoding Scheme

   Simulated Data Flow:
   Simulated CLI
   ↓
   Mock DB2 Plugin
   ↓
   Mock Database Catalog (JSON)
   ↓
   JSON Response

Simulated Output:


```
{
  "databases": [
    { "name": "CUSTDB", "owner": "SYSADM" },
    { "name": "ACCTDB", "owner": "SYSADM" }
  ]
}
```


2. **zowe db2 list tables**
   Lists tables present within a selected DB2 database.

   Real Data Flow:
   CLI
   ↓
   Zowe DB2 Plugin
   ↓
   DB2 REST Service
   ↓
   DB2 Catalog Tables
   ↓
   Table Metadata

   Data Returned:
   - Table name
   - Schema
   - Column count

   Simulated Data Flow:
   CLI
   ↓
   Mock DB2 Plugin
   ↓
   Mock Table Registry (JSON)
   ↓
   JSON Response

   Simulated Output:


```
{
  "tables": [
    { "name": "CUSTOMER", "schema": "APP", "columns": 5 },
    { "name": "ACCOUNT", "schema": "APP", "columns": 4 }
  ]
}
```

3. **zowe db2 execute query**
   Executes a SQL query on DB2 and returns the result set.

   Real Data Flow:
   CLI
   ↓
   Zowe DB2 Plugin
   ↓
   DB2 SQL Engine
   ↓
   SQL Execution
   ↓
   Result Set (Rows & Columns)

   Data Returned:
   - Rows of data
   - Column values

   This command represents actual data extraction, making it critical for data federation

   Simulated Data Flow:
   CLI
   ↓
   Mock DB2 Plugin
   ↓
   SQL Parser (Mock)
   ↓
   Mock Table Data Store
   ↓
   JSON Result Set

   Simulated Output:


```
{
  "rows": [
    { "cust_id": 101, "name": "PRITAM", "city": "BENGALURU" },
    { "cust_id": 102, "name": "ARUN", "city": "MYSURU" }
  ]
}
```


IMS Plugin – Data Flow Analysis and Simulation

What is the IMS Plugin?

The IMS plugin in Zowe enables interaction with IBM IMS (Information Management System), which is a hierarchical database and transaction management system used for high-reliability enterprise applications.

IMS differs from DB2 in that it:

- Uses **hierarchical data models**

- Manages **transactions and message queues**
- Emphasizes **availability and performance**

The Zowe IMS plugin acts as a **modern interface layer**, allowing IMS resources to be monitored and controlled using CLI commands.

General Data Flow Pattern (IMS Commands)

User Command

↓

Zowe CLI

↓

Zowe IMS Plugin

↓

IMS Connect / IMS Transaction Manager

↓

IMS Control Blocks

↓

Response (Metadata / Status)

Zowe IMS Plugin Commands:

1. **zowe ims list regions**
   Lists IMS regions and their operational status.

   Real Data Flow:
   CLI
   ↓
   Zowe IMS Plugin
   ↓
   IMS Connect
   ↓
   IMS Region Control Blocks
   ↓
   Region Metadata

   Data Returned:
   - Region name
   - Execution status (ACTIVE / INACTIVE)

Simulated Data Flow:
Simulated CLI
↓
Mock IMS Plugin
↓
Mock Region Registry (JSON)
↓
JSON Response

Simulated Output:


```
{
  "ims_regions": [
    { "region": "IMSREG1", "status": "ACTIVE" },
    { "region": "IMSREG2", "status": "INACTIVE" }
  ]
}
```


2. **zowe ims list transactions**
   Lists transaction codes managed by IMS.

   Real Data Flow:
   CLI
   ↓
   Zowe IMS Plugin
   ↓
   IMS Transaction Manager
   ↓
   Transaction Definition Tables
   ↓
   Transaction Metadata

   Data Returned:
   - Transaction ID
   - Program mapping
   - Execution status

   Simulated Data Flow:
   CLI
   ↓
   Mock IMS Plugin
   ↓
   Mock Transaction Registry (JSON)
   ↓
   JSON Response

Simulated Output:

{
"transactions": [
{ "txn_id": "IMSTXN01", "program": "CUSTIMS", "status": "ACTIVE" },
{ "txn_id": "IMSTXN02", "program": "ACCTIMS", "status": "INACTIVE"
}
]
}

3. **zowe ims start transaction:**
   Activates an IMS transaction so it can accept requests.

   Real Data Flow:
   CLI
   ↓
   Zowe IMS Plugin
   ↓
   IMS Transaction Manager
   ↓
   Transaction State Update (INACTIVE → ACTIVE)

   Data Effect:
   - No data retrieval
   - System state change

   Simulated Data Flow:
   CLI
   ↓
   Mock IMS Plugin
   ↓
   Update Transaction Status in JSON Store
   ↓
   Confirmation Response

   Simulated Output:

{
"transaction": "IMSTXN02",
"previous_status": "INACTIVE",
"current_status": "ACTIVE",
"message": "Transaction started successfully"
}

4. **zowe ims stop transaction**
   Deactivates an IMS transaction.

   Real Data Flow:
   CLI
   ↓
   Zowe IMS Plugin
   ↓
   IMS Transaction Manager
   ↓
   Transaction State Update (ACTIVE → INACTIVE)

   Simulated Data Flow:
   CLI
   ↓
   Mock IMS Plugin
   ↓
   Update Transaction Status in JSON Store
   ↓
   Confirmation Response

   Simulated Output:


```
{
  "transaction": "IMSTXN01",
  "previous_status": "ACTIVE",
  "current_status": "INACTIVE",
  "message": "Transaction stopped successfully"
}
```


Example Commands:

1. **zowe cics list regions --plex PLEX1**
2. **zowe cics list programs --region CICSREG1**
3. **zowe cics list transactions --region CICSREG1**
4. **zowe cics start program CUSTPGM --region CICSREG1**
5. **zowe cics stop program CUSTPGM --region CICSREG1**
6. **zowe db2 list databases --subsystem DB2A**
7. **zowe db2 list tables --database CUSTDB --schema APP**
8. **zowe db2 execute query "SELECT * FROM CUSTOMER"**
9. **zowe ims list regions --plex IMSPLEX1**
10. **zowe ims list transactions --region IMSREG1**
11. **zowe ims start transaction IMSTXN01 --region IMSREG1**
12. **zowe ims stop transaction IMSTXN01 --region IMSREG1**

Banking Transaction Use Case

Use Case Overview
A bank stores:

- **Customer accounts** in **DB2**
- **Transaction processing logic** in **CICS**
- **High-availability transaction routing** in **IMS**

A user wants to **transfer money and verify the result**.

STEP 0: Authentication (Once)

User (Natural Language)
"Log me into the banking system."

Agent reasoning
Secure systems require authentication before accessing banking data.

REST API ENDPOINTS
POST /auth/login

Mock response
```
{
  "token": "bank-session-789",
  "role": "bank-user"
}
```

STEP 1: User checks account balance (DB2)

User (Natural Language)
"Show me my current account balance."

Agent reasoning (WHY DB2?)

- Account balance is stored data
- Stored in relational tables
- DB2 is the correct system
- Needs a read-only query

Zowe Command Chosen
zowe db2 execute query "SELECT balance FROM ACCOUNT WHERE acc_no=12345"

REST API ENDPOINTS
POST /db2/query
Authorization: Bearer bank-session-789

Body
```
{
  "sql": "SELECT balance FROM ACCOUNT WHERE acc_no=12345"
}
```

Mock API response

{
  "rows": [
    { "balance": 25000 }
  ]
}

Final output to user
Your current balance is ₹25,000.

STEP 2: User initiates money transfer (CICS)

User (Natural Language)
"Transfer ₹5,000 from my account to account 67890."

Agent reasoning (WHY CICS?)

- Money transfer is a **real-time transaction**
- Requires **atomic execution**
- Must follow strict consistency rules
- CICS is designed for **online banking transactions**

Zowe command chosen
zowe cics start program TRANSFERPGM --region CICSBANK1
(TRANSFERPGM = banking transfer application)

REST API ENDPOINTS
POST /cics/regions/CICSBANK1/programs/TRANSFERPGM/start
Authorization: Bearer bank-session-789

Mock API response

{
  "program": "TRANSFERPGM",
  "status": "ENABLED"
}

Final output
Transaction processing program activated.

STEP 3: CICS processes the transaction
Internal agent action
CICS program:

- Debits ₹5,000 from source account
- Credits ₹5,000 to destination account
- Ensures rollback if failure occurs

(No direct Zowe command here — handled inside CICS)

STEP 4: Verify transaction status (IMS)

User (Natural Language)
“Did my transfer go through successfully?”

Agent reasoning (WHY IMS?)

- IMS handles transaction routing and status
- Used for high-availability confirmation
- Ensures transaction lifecycle tracking

Zowe command chosen
zowe ims list transactions --region IMSBANK1

REST API ENDPOINTS
GET /ims/regions/IMSBANK1/transactions
Authorization: Bearer bank-session-789

Mock API response


```
{
 "transactions": [
   {
    "txn_id": "TXN9001",
    "type": "TRANSFER",
    "status": "COMPLETED"
   }
 ]
}
```


Final output
Your transfer of ₹5,000 was completed successfully.

STEP 5: Updated balance check (DB2)

User (Natural Language)
“Show my updated balance.”

Agent reasoning
Balance is stored data → DB2 query required again.

Zowe command
zowe db2 execute query "SELECT balance FROM ACCOUNT WHERE acc_no=12345"

Mock response


```
{
 "rows": [
   { "balance": 20000 }
 ]
}
```


Final output
Your updated balance is ₹20,000.

HOW ZOWE INTERPRETS NATURAL LANGUAGE
Zowe itself **does not understand English**.

User (English)
↓
AI / Agent Layer (Intent Detection)
↓
System Decision (DB2 / CICS / IMS)
↓
Zowe CLI Command
↓
REST API
↓
Mainframe Subsystem